

CbcSolver: Class Architecture Overview

coin-or/Cbc — next branch — src/CbcSolver.hpp / src/CbcSolver.cpp

What is CbcSolver?

CbcSolver is the top-level driver class for the Cbc MIP solver executable and library entry point, encapsulating the entire solve pipeline — parameter initialization, command-queue parsing, LP relaxation solves, preprocessing, cut/heuristic configuration, branch-and-bound search, post-processing, and result reporting — that used to live in the free functions `CbcMain0/CbcMain1`, now thin backward-compatible wrappers delegating to `initialize()` and `run()`.

Historically, `CbcSolver::run()` was a single ~14,000-line function built around one giant `switch` on the current command. A series of refactors (most recently: removing dead `#ifndef JJF_ONE / #if 0` blocks, splitting the branch-and-bound `case` into six cooperating private methods, and deleting the legacy hard-coded `-miplib/-unitTest` self-test harness) progressively extracted coherent phases into named methods listed below, while `run()` itself was reduced to a slim command dispatcher. Local variables that needed to survive across these phases were promoted to `private` data members (see §4).

1 Public API surface

- **Construction & lifecycle:** default / from-solver / from-model / copy constructors, `operator=`, destructor (lines 1927–2341); `initialize()` sets up signal handlers and default parameters (replaces `CbcMain0`).
- **High-level solve:** `solve(file)`, `importModel(file)`, `solveLp(method)` — convenience one-call wrappers that chain `initialize()` + `run()` or solve the LP relaxation directly.
- **Result accessors:** `status()`, `objectiveValue()`, `bestBound()`, `bestSolution()`, `numCols()`, `nodeCount()`, `iterationCount()`, `hasSolution()` — valid after `solve()/run()` returns.
- **Run (replaces CbcMain1):** `run(argc,argv,...)` delegates to `run(deque,...)`, the primary command-queue-driven entry point and the root of the pipeline in §2.
- **Heuristic / cut-generator configuration:** `configureHeuristics()`, `configureCutGenerators()` — public wrappers exposing internal BAB setup so callers can invoke it directly on a model without going through `run()`.
- **User extensions:** `addUserFunction()`, `setUserCallBack()`, `addCutGenerator()` register `CbcUser` / `CbcStopNow` / `CglCutGenerator` plug-ins.
- **Accessors:** `model()`, `babModel()`, `parameters()`, `intValue()` / `setIntValue()`, `doubleValue()` / `setDoubleValue()`, `originalSolver()`, `originalCoinModel()`, `getStatistics()`, and similar simple getters/setters over the state in §4.

2 The branch-and-bound pipeline

`run()` dispatches on the current `CbcParam` command. Besides simple one-shot actions (§3), the **BAB** command drives the solver's core pipeline, implemented as six cooperating private methods called in sequence (see the full diagram on the next page):

1. `babSetupAndRootLp()` — cutoff sign flip, legacy `OsiSolverLink` path, solves the root LP via `solveInitialLp()` → `applyLpMethod()`.
2. `babConfigureBabModel()` — (re)builds `babModel_` from `model_`, configures the conflict-graph branching ranker (`RankerConfig`), tunes factorization frequency.
3. `preprocess()` — runs `CglPreProcess` when preprocessing is enabled.
4. `babPostPreprocessCleanup()` — cgraph/cliq-strengthening cleanup, post-preprocess bound tightening.
5. `babConfigureSearchModel()` — knapsack expansion, cost-based priorities, and invokes `configureHeuristics()` / `configureCutGenerators()`.
6. `babExecuteSearchAndPostprocess()` — SOS/priority setup, the actual `babModel_→branchAndBound()` call, search-statistics bookkeeping, and finally `postprocess()` to translate the solution back and report results.

Each phase returns a status code consumed by `run()` to decide whether to continue, **break** the BAB case, or **return** from `run()` outright — preserving the exact control flow of the original monolithic function.

3 Other run() actions

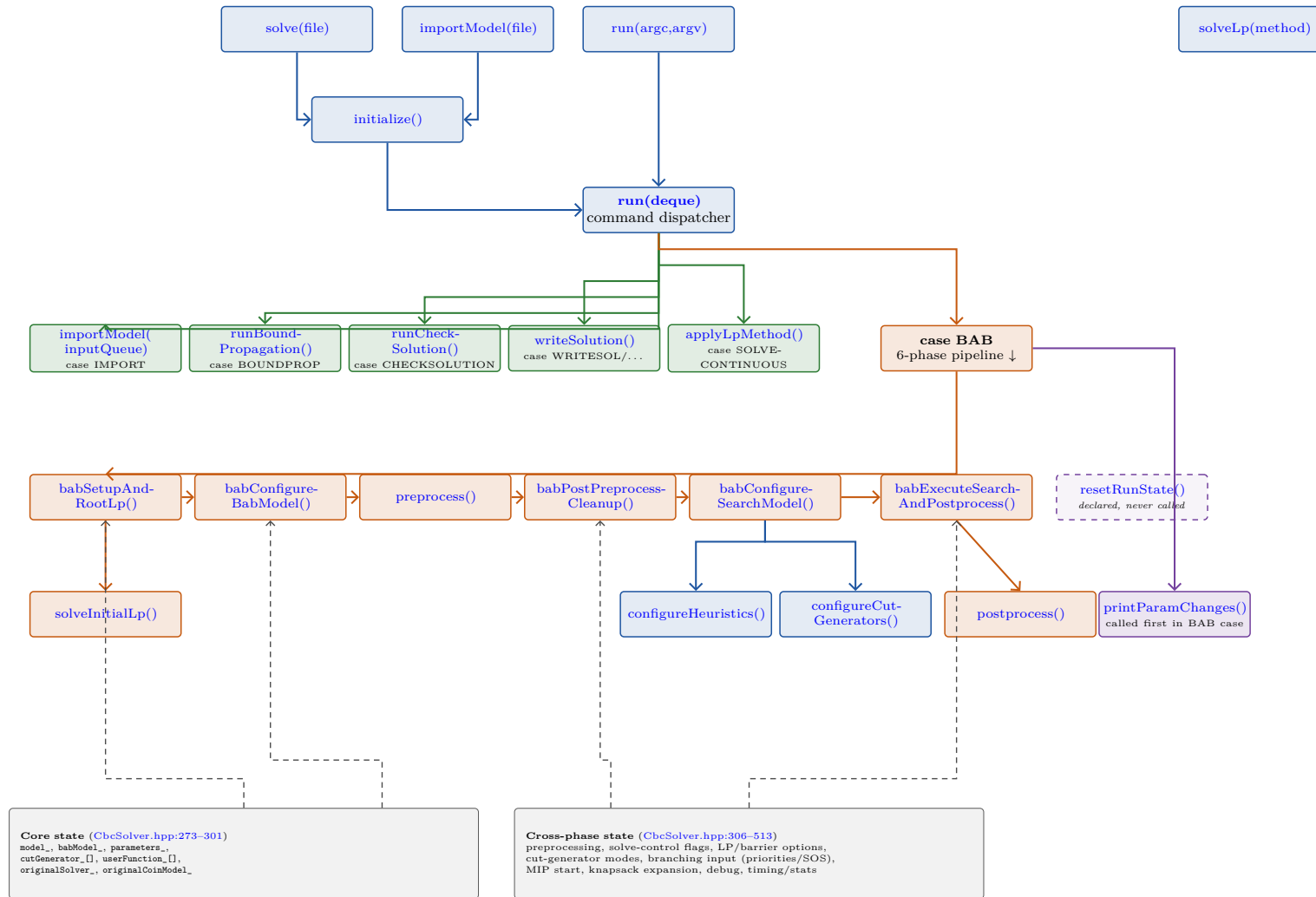
- **IMPORT:** private `importModel(inputQueue,...)` reads an MPS/LP/GMPL file.
- **SOLVECONTINUOUS:** `applyLpMethod()` solves the LP relaxation only.
- **BOUNDPROP:** `runBoundPropagation()` runs standalone bound propagation.
- **CHECKSOLUTION:** `runCheckSolution()` writes a solution-validation report.
- **WRITESOL / PRINTSOL / ...:** `writeSolution()` writes solutions in various formats.
- **Bookkeeping:** `printParamChanges()` prints a parameter-change banner at the top of the BAB case; `resetRunState()` is declared/defined but currently *never called* (orphaned — dashed outline in the diagram).

4 State organization

Private data falls into two groups. **Core state** (`model_`, `babModel_`, `parameters_`, `cutGenerator_`, `userFunction_`, ...) existed in the original class. **Cross-phase state** (lines 306–513) — preprocessing (`process_`, `saveSolver_`), solve-control flags, LP/barrier options, cut-generator modes, branching input (priorities/SOS/pseudo-costs), MIP start, knapsack-expansion buffers, debug, lot-sizing, and timing/statistics — used to be *local variables* inside the monolithic `run()`. Promoting them to member fields is what let `run()` be split into the cooperating methods of §2: each extracted method reads/writes the subset of this state it needs directly, instead of receiving dozens of by-reference parameters.

Colour legend used on the diagram (next page; node fill and arrow colour share the same code, so each call arrow is coloured like the method it leaves): ■ public API ■ BAB pipeline (§2) ■ other `run()` actions ■ bookkeeping helpers ■ state / data structures (dashed = data reads/writes)

CbcSolver — method call graph & state ownership (node labels are hyperlinks to `coin-or/Cbc@next`)



Arrows show call/return relationships within the modularized BAB pipeline (§2); dashed grey arrows indicate representative reads/writes of the shared state panel on the left (most pipeline methods touch several state groups — only the busiest links are drawn to keep the diagram readable). All labels link to the corresponding line in `coin-or/Cbc`, branch `next`.